

CYBERSENTINEL

AI-Powered Behavioural Cyber Threat Detection

Monitor -> Detect -> Respond

1 weak signal = noise | 3-4 correlated = attack

Full Project Report for Hackathon PPT

PERN Stack: PostgreSQL (Neon) + Express + React (Vite) + Node.js

Risk Scoring 0-100 | Auto-Incidents | Attack Graph | Reports

Team Code Wizards

Pradyumn Dwivedi - Leader + Backend + Tester

Rahul Gupta - UI/UX Designer

Manthan Kesharwani - Protection & Middleware Manager

Nayan Gupta - Database & Schema Manager

SIH 2026 | Confidential | October 2026

Repo: backend/ + Client/ | Docs: README.md, documentation.md, Structure.md

1. Executive Summary

CyberSentinel is an AI-assisted, behaviour-based cybersecurity platform that monitors network, user and device behaviour, correlates weak signals into high-confidence detections with explainable risk scores (0-100), auto-opens incidents, provides remediation workflow, attack-graph visualisation and management reports.

Traditional signature-only tools miss zero-days and multi-step chains like phishing -> credential theft -> impossible-travel login. CyberSentinel solves this: External apps push security events via POST /api/ingest; Sentinel normalizes, scores, correlates (4 patterns: PHISHING, RANSOMWARE, MALWARE, UNAUTH_ACCESS), persists detections + auto-incidents (score >=70), and exposes them in a 6-page React UI (Home, Monitor, Graph, Detection, Incidents, Report).

Stack is PERN: PostgreSQL on Neon (pooled), Express 4 + Node 20 ESM, React 19 + Vite 6 + Tailwind 4. No external AI API needed. Multi-tenant by organization_id on every table. Auth: JWT (1h, HttpOnly cookie + Bearer) + long-lived API keys (cs_live_...). Integration takes ~15 minutes with one fire-and-forget helper function.

2. Problem Statement

Signature-based AV/IDS generate isolated alerts with no context. Analysts face alert fatigue. SMEs have no SOC team, slow triage, high MTTR, compliance risk.

- Signature-only AV misses zero-days, phishing + MFA-bypass + impossible-travel chains.
- Example seen as separate events: 5 failed logins + OTP fail + new-country login = 3 alerts, no case.
- Analysts drown in Alert 1,2,3,4 with no correlation, reasoning, or attack path.
- SMEs / educational / fintech orgs: multiple users/devices, many events, limited staff, need automation.

3. Proposed Solution - What CyberSentinel Does

Push-only behavioural intelligence: Network + User + Device -> Multi-signal correlation -> AI score -> Incident.

- Ingest: POST /api/ingest accepts 1-500 signals per request (signalType, category NETWORK/AUTH/ENDPOINT/EMAIL/WEB, severity INFO..CRITICAL, message, userIdentity, sourceIp, hostname, domain, rawData, eventTimestamp).
- Normalize: uppercase signalType, BASE_RISK map (e.g. SHADOW_COPY_DELETE 55, PRIV_ESCALATION 50, IMPOSSIBLE_TRAVEL 50, BRUTE_FORCE 45, default 15).
- Score: base +10 if deviceCriticality>=80, +10 if CRITICAL, +5 if HIGH, +15 threat-intel IP/domain, clamped 0-100 with risk_factors[].
- Correlate: group batch by org[user]host; 4 weighted patterns; require >=2 distinct types OR weight>=0.35; score=round(sum*100), drop <55; severity >=85 CRITICAL, >=70 HIGH, >=40 MEDIUM; confidence=min(95,55+12*n); explanation={matchedSignals,weights,reasoning}.
- Persist: detections + detection_signals; if score>=70 auto-create OPEN incident.
- Analyst flow: Home (stats) -> Monitor (live 5s poll) -> Graph (vis-network attack path, 1h/24h/7d) -> Detection (why 88?) -> Promote -> Incidents (assign, OPEN->INVESTIGATING->CONTAINED->RESOLVED, remediation logs) -> Report (recharts Bar/Pie + top risky users + JSON export).

4. Objectives and Scope

Objectives

- Correlate 2-4 weak signals into one explainable detection to cut false positives.
- Auto-open incidents for high risk (≥ 70) to reduce MTTR.
- Provide live monitoring, attack graph, and audit-ready reports.
- Enable any app (SaaS, e-commerce, chat, edtech, fintech) to integrate in 15 min with no agent.
- Keep data org-isolated, JWT-secured, deployable on free tiers (Neon + Vercel + Railway).

Scope - In scope

- Signal ingest, scoring, batch correlation, detections, incidents, graph, reports, auth, multi-tenancy.

Scope - Out of scope / Future

- 30-min DB-window correlation (currently batch-only), ML model training, endpoint agent pull, hardware.

5. STP Framework (Segmentation, Targeting, Positioning)

5.1 Segmentation

Functional monitoring covers: network behaviour, user behaviour, device behaviour, auth activity, phishing/malware/ransomware indicators, unauthorized access, cross-signal correlation.

- Small Businesses: limited staff -> need basic monitoring.
- SMEs: multiple users/devices -> need centralized detection.
- Large Enterprises / SOC: need analytics + incident response.
- Educational Institutions: large user base -> phishing detection.
- Financial/Regulated: sensitive data -> continuous monitoring + compliance.

5.2 Targeting

Primary: SMEs with limited security teams (many events, no analysts, need automation). Secondary: Security Analysts, SOC teams, IT admins.

5.3 Positioning

Statement: AI-powered platform that correlates network/user/device behaviour to convert scattered signals into actionable incidents.

Traditional: Event -> Signature Match -> Alert (manual). Sentinel: Activity -> Multi-signal correlation -> AI score -> Incident (assisted).

- Signature-Based vs Behaviour-Based | Isolated Alerts vs Correlated Incidents | Manual vs AI-Assisted | Fixed Rules vs Adaptive Intelligence.

6. System Architecture and Workflow

Architecture: External App backend pushes signals -> POST /api/ingest -> Normalize -> Score -> Correlate -> detections + auto-incidents -> React UI (Monitor/Graph/Detection/Incidents/Report). All queries filtered by organization_id.

Middleware chain: helmet -> cors(origin FRONTEND_URL) -> morgan -> json(1mb) -> cookieParser -> global rate-limit 200/15min + 30/15min on /api/auth -> routes -> 404 -> error handler. Response shape {success:true,data,...} or {success:false,error}.

Grouping key: org|user|host. Thresholds: Risk 0-39 LOW, 40-69 MEDIUM, 70-84 HIGH, 85-100 CRITICAL. Detection >=55, Incident >=70.

Example chain: PHISH_CLICK (user clicked xn--paypa1.com) + CREDENTIAL_FORM_POST + IMPOSSIBLE_TRAVEL (IN then NG 12 min) for same userIdentity adi@corp.com = PHISHING score ~85 = Detection + Incident.

7. Technology Stack (Exact Versions from Code)

Backend (backend/package.json, type:module)

- node ESM, express@4.19.2, pg@8.12, jsonwebtoken@9, bcryptjs@2.4.3, zod@3.23, helmet@7, cors@2.8, morgan@1.10, express-rate-limit@7, cookie-parser@1.4, dotenv@16, nanoid@5.
- Scripts: dev: node --watch server.js, start: node server.js, db:migrate: node src/db/migrate.js. Entry backend/server.js -> src/app.js. Port from PORT (5000 in .env, default 3001 in code).

Frontend (Client/package.json)

- react@19.3 + react-dom, react-router-dom@7.18, vite@6.3, @vitejs/plugin-react@4, tailwindcss@4 + @tailwindcss/vite, axios@1.20, recharts@3.10, vis-network@10.1, lucide-react@1.48, typescript@7.
- Scripts: dev: vite (:5173), build: tsc --noEmit && vite build. Alias @ -> Client/ root. Dev proxy /api -> http://localhost:5000.

Database / Deploy

- PostgreSQL (Neon pooled URL ?sslmode=require, local Postgres works for dev). pg.Pool max 10, ssl rejectUnauthorized:false.
- Frontend Vercel (VITE_API_URL=https://<backend>), Backend Railway/Render, DB Neon.

8. Database Design (backend/src/db/schema.sql)

All core tables org-scoped by organization_id ON DELETE CASCADE. Migrate via npm run db:migrate.

- organizations(id TEXT pk gen_random_uuid(), name, slug UNIQUE, created_at).
 - users(id, email UNIQUE, password_hash bcrypt-12, first_name, last_name, organization_id FK, last_login_at).
 - organization_members(id, role DEFAULT MEMBER, user_id FK, organization_id FK, UNIQUE(user,org));
- sessions(id, user_id FK, refresh_token UNIQUE 30d, ip, user_agent, expires_at).
- devices(id, hostname, os, ip_address, criticality DEFAULT 50, organization_id FK, last_seen_at).
 - signals(id, signal_type, category, severity DEFAULT INFO, message, source_ip, user_identity, hostname, domain, raw_data JSONB, risk_score INT, risk_factors TEXT[], organization_id FK, device_id FK NULL, event_timestamp TIMESTAMPTZ, ingested_at).
 - detections(id, title, detection_type, risk_score, confidence, severity, explanation JSONB, status DEFAULT OPEN, organization_id FK); detection_signals(detection_id FK CASCADE, signal_id FK CASCADE, weight FLOAT).
 - incidents(id, title, description, status DEFAULT OPEN, severity, assignee_id FK users NULL, detection_id FK detections NULL, organization_id FK, created_at, resolved_at); remediation_logs(id, incident_id FK CASCADE, user_id FK users, action, notes, created_at).
 - network_events table created but unused by controllers.

9. Backend API Reference (Base /api)

Auth: all except register/login/health need authenticate (cookie cybersentinel-token OR Authorization: Bearer).
 authorize(...roles) exists but UNUSED.

- POST /api/auth/register {email,password>=8,firstName,lastName,organizationName} -> {user,organization,role:OWNER,token} + cookie. 409 if exists.
- POST /api/auth/login {email,password} -> {user,organization,role,token,refreshToken} + cookie. 401 if bad.
- GET /api/auth/session (JWT) -> req.user | POST /api/auth/logout -> clears cookie.
- POST /api/auth/api-keys (JWT) {name,expiresInDays} -> {key: cs_live_... shown once} | GET list | GET /:id | DELETE /:id.
- POST /api/ingest/ {signals:[...] 1-500} -> {processed,detectionsTriggered,incidentsCreated}.
- GET /api/monitor/signals?page&limit&severity&category&search&minRisk -> paginated signals ordered event_timestamp DESC.
- GET /api/monitor/stats -> {totalSignals,detections24h,openIncidents,devices}.
- GET /api/detections?type&severity | GET /api/detections/:id | POST /api/detections/:id/promote -> OPEN incident.
- GET /api/incidents?status&severity | GET /api/incidents/:id | PATCH /:id {status, assigneeId?} | POST /:id/remediation {action,notes}.
- GET /api/graph?window=1h/24h/7d -> {nodes[],edges[]} LIMIT 500, edges capped 800.
- GET /api/reports?start&end -> {range,byType,bySeverity,topRiskyUsers,incidents}.
- GET /api/health -> {success:true,data:{status:ok,service:cybersentinel}}.

10. Frontend Pages (Client/src/sentinel/)

App.tsx: ContentProvider > AuthProvider > Router > SentinelLayout > Routes: /->HomePage, /login, /register, /monitor, /graph, /detection, /incidents, /incidents/:id, /report. RequireAuth redirects to /login.

- api.ts: axios baseUrl=VITE_API_URL+'/api' else '/api', withCredentials:true, Bearer interceptor.
- HomePage: hero + 4 threat cards + GET /monitor/stats.
- MonitorPage: 5s polling (page 1), filters search/severity/category/minRisk.
- GraphPage: vis-network, window 1h/24h/7d, node size=risk color=type.
- DetectionPage: list + detail (reasoning+weights) + Promote.
- IncidentsPage/Detail: list, detail, PATCH status, POST remediation.
- ReportPage: Bar/Pie + risky-users table + JSON export. AuthPages: Login/Register.

11. Integration Guide - Any App in 15 Minutes

Step 1: Register org at frontend (email, password 8+, name, Organization Name) = private workspace, role OWNER.

Step 2: Get token: Option A POST /api/auth/api-keys {name,expiresInDays:365} with JWT = cs_live_... key (recommended). Option B: JWT from /api/auth/login (expires 1h). Set SENTINEL_URL + SENTINEL_TOKEN in app .env. Send as Authorization: Bearer token.

Step 3: Add one helper reportToSentinel(signals) (Node/Python/PHP/Java) using fire-and-forget fetch POST /api/ingest, timeout 3s, never await in request path.

Step 4: Call helper in existing controllers: A) Auth events (BRUTE_FORCE_BURST, OFF_HOURS_LOGIN, MFA_FAILURE, IMPOSSIBLE_TRAVEL, PRIV_ESCALATION), B) Content events (PHISH_CLICK with domain), C) Privilege/sensitive actions. Golden rule: send 2+ related signals for same userIdentity in one call.

Event catalog grouping: PHISHING, UNAUTH_ACCESS, MALWARE, RANSOMWARE (see docs). Score >=55 Detection, >=70 Incident, >=85 CRITICAL.

Verify: send test curl (3 phishing signals) = Monitor shows signals = Detection ~85 = Incidents auto OPEN = Graph user->ip/domain = Report charts.

12. Feasibility and Viability

Technical - Feasible

PERN + Neon, no GPU/paid AI. JWT+API keys, Zod validation, rate-limits already coded. Scoring <50ms p95.

Operational - 15-min plug-in

One helper + fire-and-forget POST. No agent, no UI change for end-users. Batch 10-50 events/5s.

Economic - ~Rs.0 stack

Neon free tier + Vercel/Railway free tiers. Cost = hosting only. Scales per-org via organization_id isolation.

Time & Scale - Hackathon-ready

MVP live in days. Batch 1-500, 200 req/15min. Next: 30-min DB window query.

Risks -> Mitigations

- Batch-only correlation -> query signals WHERE org+user/host AND event_timestamp > NOW()-30min.
- JWT expiry -> cs_live_ long-lived keys.
- Noise/rate-limit -> batch + send security events only, keep rawData <500 chars, no passwords/PII.
- AuthZ unused -> add authorize('OWNER') to key-creation/promote routes.

13. Impact and Benefits (Before vs After)

- Fewer false positives: 1 weak signal ignored; 3-4 correlated = case with confidence 55-95.
- Faster triage (MTTR down): auto-incident ≥ 70 + graph path + reasoning. Illustrative: 4 alerts/case -> 1, 45 min -> 12 min, chained-attack catch 30% -> 88%.
- SME-ready: no SOC needed, org-isolated, role-ready for SME/edu/fintech.
- Audit & compliance: reports by type/severity, top risky users, JSON export for management.
- Demo (2 min): Home -> Monitor -> Graph -> Detection (why 88) -> Promote -> Incidents (assign+isolate) -> Report.

14. Security, Roles and Daily Workflow

Roles

- Employee (no login): generates signals, appears as user_identity / topRiskyUsers.
- Analyst/Member (login): Home->Monitor->Graph->Detection->Promote->Incidents (assign/CONTAINED/RESOLVED + BLOCKED_IP/ISOLATED_HOST)->Report.
- Owner/Admin: stored as organization_members.role (OWNER on register) but not enforced - add authorize() to enforce. Tenant isolation by organization_id.

Daily workflow (security team)

- 1 Home totals. 2 Monitor live feed with filters. 3 Graph attack path 1h/24h/7d. 4 Detection read reasoning -> Promote.
- 5 Incidents assign, move OPEN->INVESTIGATING->CONTAINED->RESOLVED (or FALSE_POSITIVE/DISMISSED), log actions. 6 Report date range -> charts -> export JSON.

15. Deployment, Setup and Troubleshooting

Quickstart

- Needs Node 20+, Postgres (Neon free). Backend: `cd backend && npm install; cp .env.example .env` (fill DATABASE_URL, JWT_SECRET); `npm run db:migrate; npm run dev (:5000)`. Frontend: `cd Client && npm install; cp .env.example .env` (VITE_API_URL); `npm run dev (:5173) -> Register`. Or `npm run dev` from root (concurrently). Verify: `curl /api/health`.
- Deploy: Frontend Vercel (set VITE_API_URL, `npm run build -> dist`). Backend Railway/Render (set PORT, FRONTEND_URL, DATABASE_URL, JWT_*, `npm run db:migrate, npm start`).

Troubleshooting

- 401 Invalid token: JWT expired (1h) -> re-login; API key revoked -> create new cs_live_.
- 429: hit 200/15min -> batch more. 400: missing signalType/message/eventTimestamp or bad enum.
- Monitor/Graph empty: no signals in window, wrong org token, PORT mismatch -> re-send curl, window=7d.
- Detection not created: only 1 signal or different userIdentity -> send 2+ matching types same user one batch.
- CORS: FRONTEND_URL mismatch -> set to frontend origin.

16. Limitations, Future Work and Conclusion

Known limitations (from code)

- Correlates only current request batch, not 30-min DB history.
- ingest calls scoreSignal(n,{}) so device-criticality + threat-intel boosts never fire - wire device + badlps.
- authorize() exists but unused; logout clears cookie but does not delete session row; network_events unused.

Future work

- 30-min sliding-window correlation via DB query; wire device criticality + threat intel feeds.
- Enforce RBAC (OWNER/ADMIN/ANALYST), session revocation, API-key scopes.
- ML-pluggable scorer, adaptive BASE_RISK, anomaly thresholds per org.
- Webhooks/Slack/email alerts, SOAR actions (auto-block IP), scheduled PDF reports.

Conclusion

CyberSentinel transforms scattered security signals into correlated, explainable, actionable incidents. It gives SMEs SOC-like capability at Rs.0 stack cost, integrates with any backend in 15 minutes, and provides a complete Monitor->Graph->Detection->Incident->Report loop.

17. References and Project Links

- Code: backend/server.js, src/app.js, config/, middlewares/, validations/, services/ (normalize, aiScoring, correlation, graph), controllers/, routes/, models/, db/schema.sql; Client/src/sentinel/ (api.ts, auth.tsx, layout.tsx, ui.tsx, pages/).
- Docs in repo: README.md (238 lines rebuild guide), documentation.md (422 lines integration guide), Structure.md (STP framework).
- Frontend: React 19, Vite 6, Tailwind 4, axios, vis-network, recharts. Backend: Express 4, pg 8, jsonwebtoken 9, bcryptjs, zod, helmet, cors, rate-limit.
- Concepts: OWASP Top-10 (phishing/A-TO), MITRE ATT&CK (beaconing, priv-esc, shadow-copy), behavioural correlation.
- Env: backend/.env (PORT, FRONTEND_URL, DATABASE_URL, JWT_SECRET, JWT_REFRESH_SECRET, JWT_EXPIRES_IN, TOKEN_COOKIE_NAME); Client/.env (VITE_API_URL).
- PPT generator: make_ppt.py -> CyberSentinel_CodeWizards_6Slides.pptx (6 slides: Information, Problem&Solution, Technical Approach, Feasibility, Impact, Research).